

Transformer

Language Models

Course Contents

Deep Learning

Transformer

LLMs

Context Engineering

Finetuning

Instruction Tuning

Agents

The Bigger Picture

This Part: Transformer

- tokenizer
- static embeddings
- contextual embeddings
- text generation

Tokenization

tokenization: *breaking text in chunks*

- word tokens: different forms, spellings, etc → undefined and vast vocabulary
- character tokens: not enough semantic content (longer sequences)
- popular compromise: byte-pair encoding

Byte-Pair Encoding

data compression method used for encoding text as sequence of tokens (sub-words)

- merging token pairs (starting with characters) with maximum frequency
 - continue merging until defined fixed vocabulary size (hyperparameter) is reached
- common words encoded as single token
- rare words encoded as sequence of tokens (representing word parts)

alternative: direct operation on bytes (e.g., [ByT5](#))

aaabdaaabc

ZabdZabac
Z=aa

ZYdZYac
Y=ab
Z=aa

XdXac
X=ZY
Y=ab
Z=aa

example from wikipedia

Embedding Layers

representation of entities by vectors

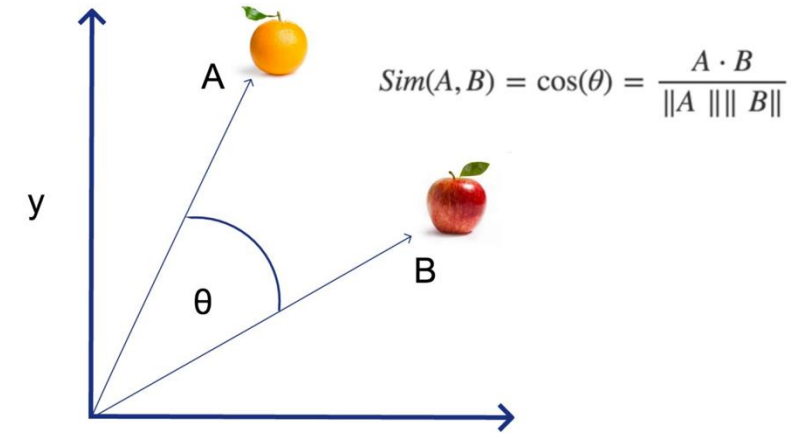
→ semantic similarity

most famous application: word embeddings

→ associations (natural language processing)

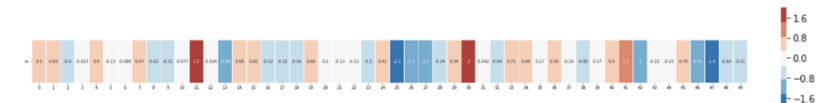
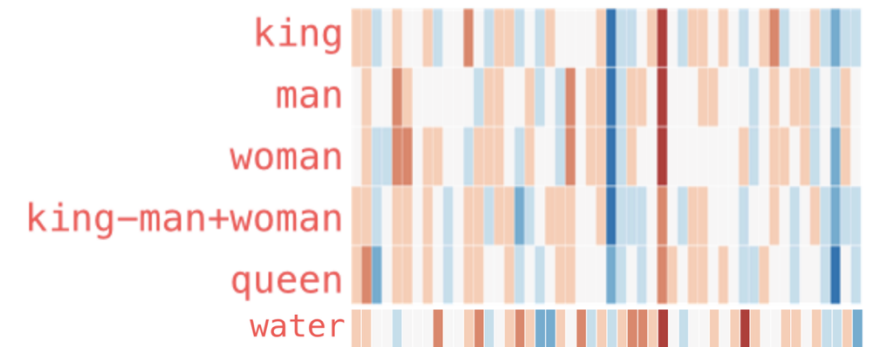
but general concept: embeddings of
(categorical) features (e.g., products in
recommendation engines)

learned via co-occurrence (e.g., [word2vec](#))



but also direction of difference
vectors interesting (analogies):

king - man + woman $\approx$ queen



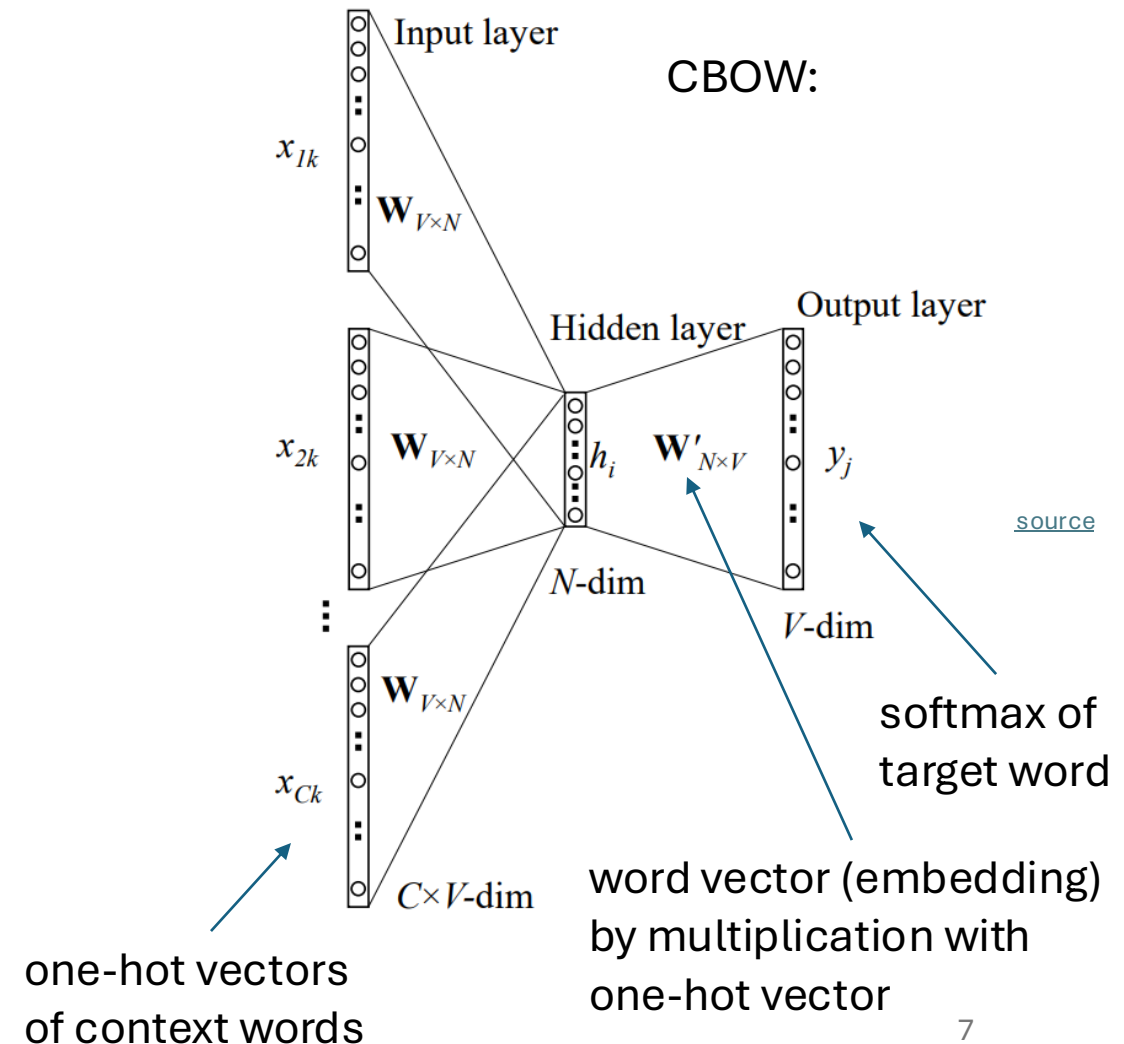
[source](#)

Some Thoughts on Word Embeddings

can be implemented as

- neural network with single hidden layer (linear activation)
- using, e.g., bag-of-words approach (predict masked word from its surroundings)

→ not context-aware



Word Embeddings as Part of Language Model

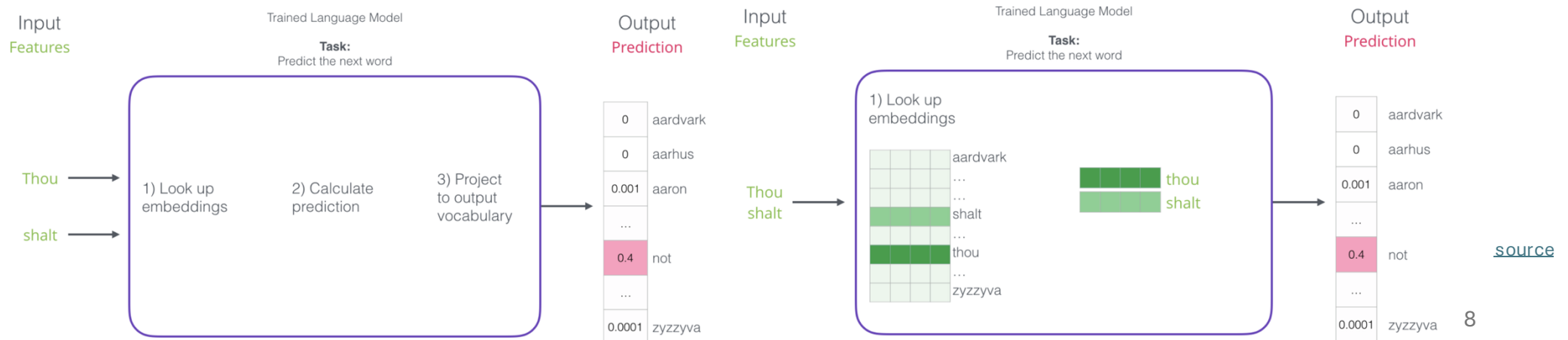
language models contain embedding matrix as part of learned parameters

typically several hundred dimensions for word vectors (to be compared with vocabulary sizes of many thousands)

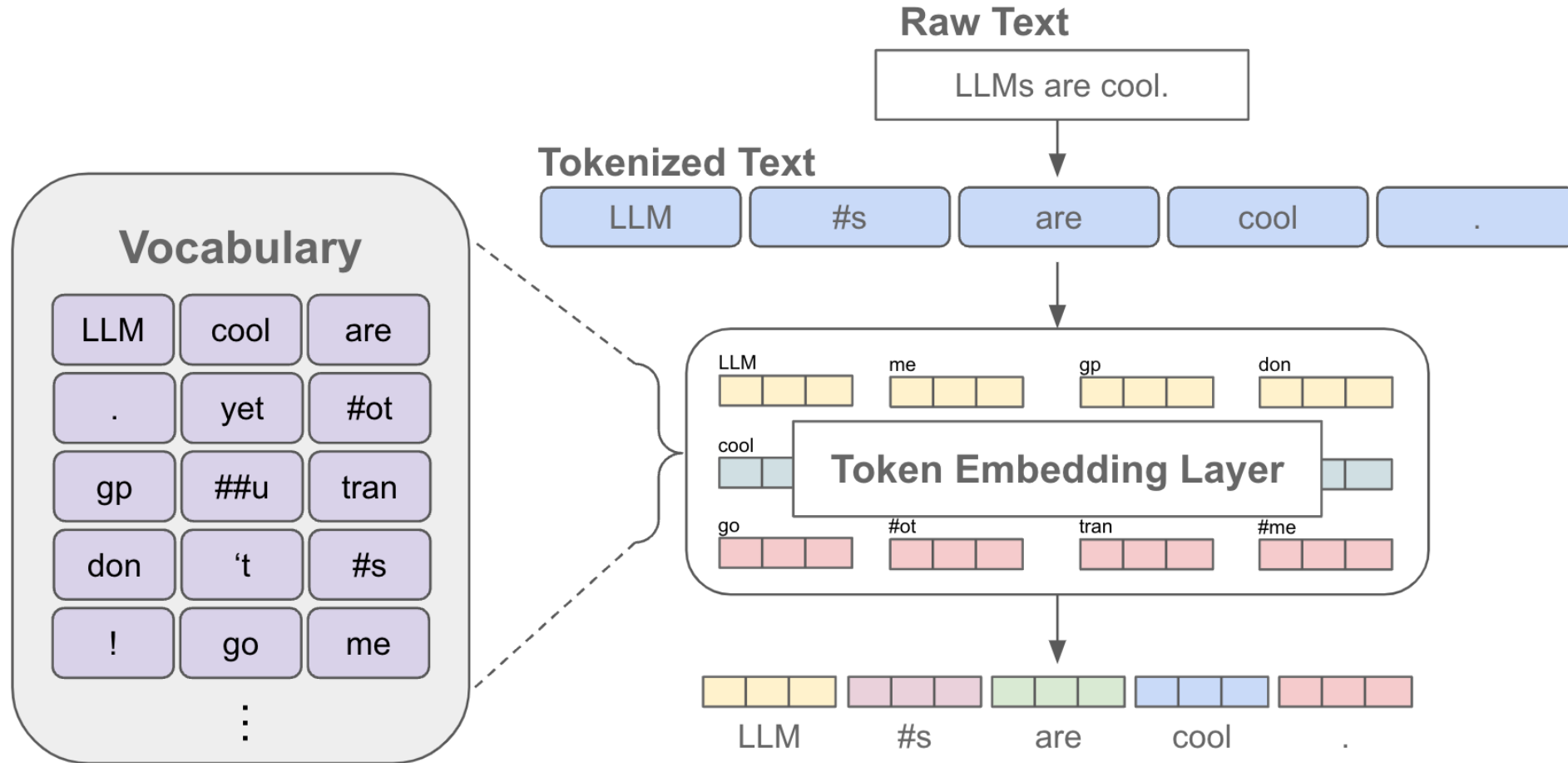
(can also be extracted and subsequently used as pretrained embeddings for other task)



next-word
prediction:



Tokenization & Embeddings



[source](#)

Self-Supervised Learning

unsupervised learning (learning by observation)

no target information → kind of “vague” pattern recognition (but plenty of data)

can be cast as **self-supervised learning**:

- input-output mapping like supervised learning
- but generating labels itself from inputs

The Lord of the Rings is considered one of the greatest

The Lord of the Rings is considered one of the greatest fantasy

ML needs lots of training data

A look at unsupervised learning

■ “Pure” Reinforcement Learning (cherry)

- ▶ The machine predicts a scalar reward given once in a while.

▶ **A few bits for some samples**

■ Supervised Learning (icing)

- ▶ The machine predicts a category or a few numbers for each input
- ▶ Predicting human-supplied data
- ▶ **10→10,000 bits per sample**

■ Unsupervised/Predictive Learning (cake)

- ▶ The machine predicts any part of its input for any observed part.
- ▶ Predicts future frames in videos
- ▶ **Millions of bits per sample**

■ (Yes, I know, this picture is slightly offensive to RL folks. But I’ll make it up)

Original LeCun cake analogy slide presented at NIPS 2016, the highlighted area has now been updated.

[source](#)



Context Awareness: Transformer

Attention Is All You Need:

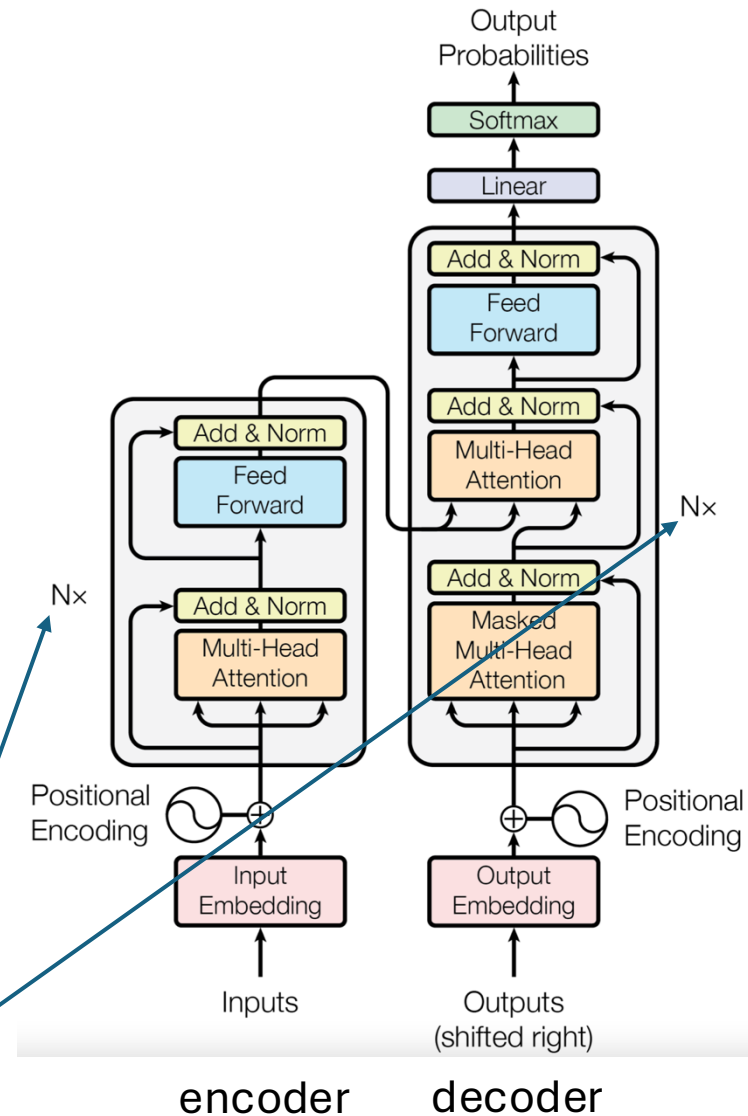
completely replaced recurrent neural networks (RNN)

much more parallelization → bigger models

better long-range dependencies thanks to shorter path lengths in network (less sequential operations)

encoder/decoder stacks (depth):
output fed as input to next one

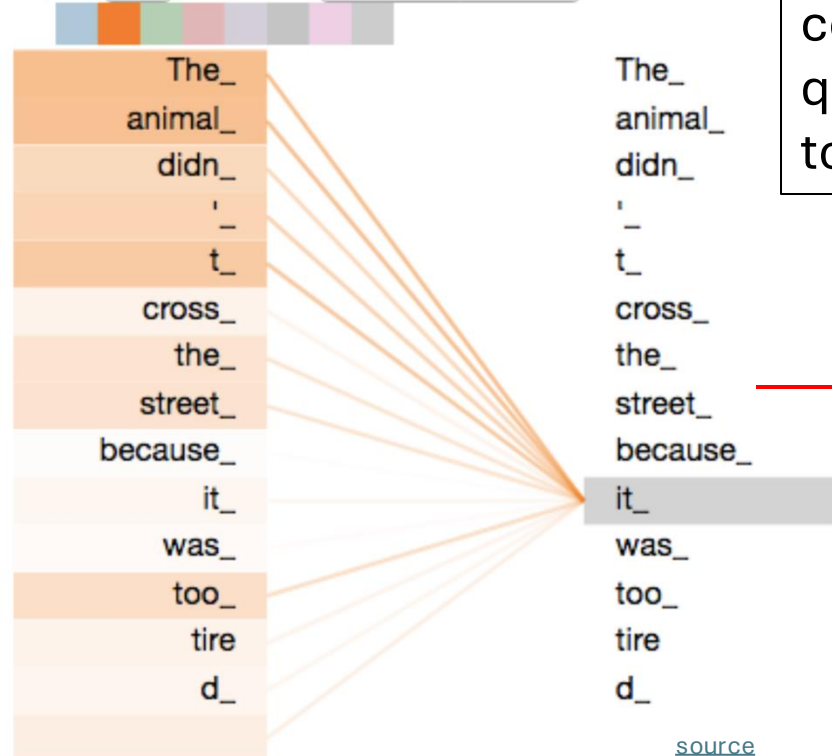
original transformer:
sequence-to-sequence model
(e.g., for machine translation)



Self-Attention Mechanism

evaluating other input tokens in terms of relevance for encoding of given token

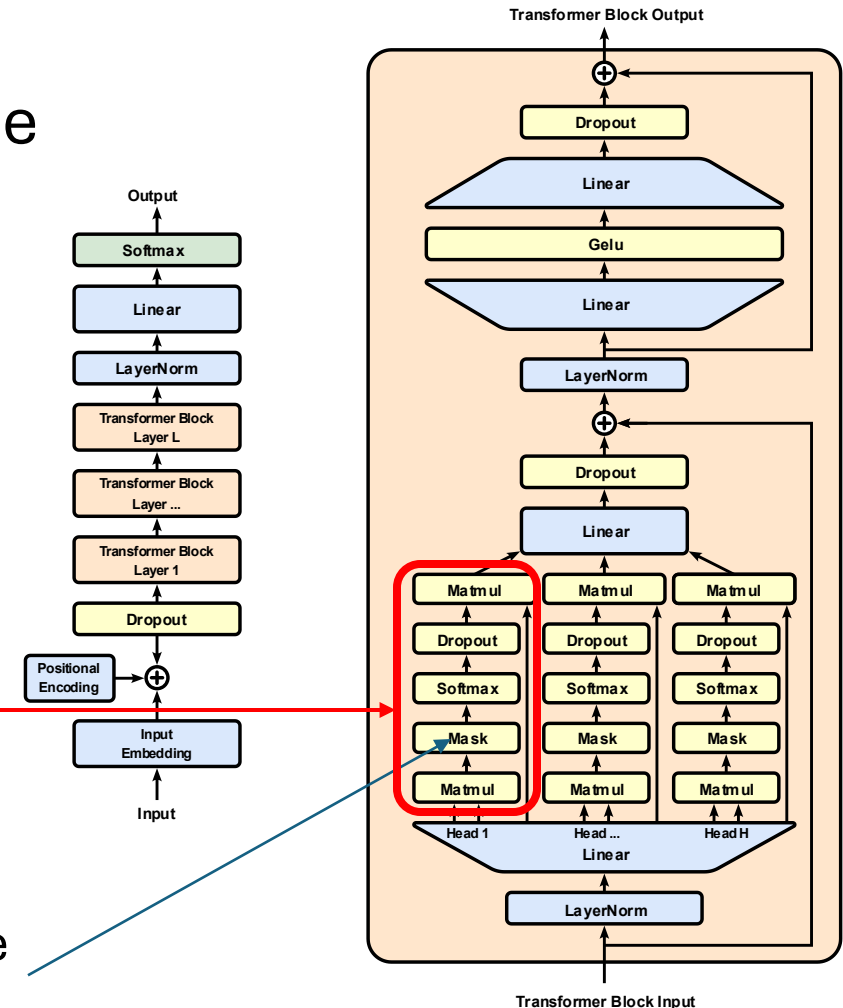
Layer: 5 Attention: Input - Input



computational complexity
quadratic in length of input (each
token attends to each other token)

only allow to attend to
earlier positions in sequence
(masking future positions by
setting them to $-\infty$)

GPT (decoder-only):



Scaled Dot-Product Attention

softmax not scale invariant: largest inputs dominate output for large inputs (more embedding dimensions d_k)

3 matrices created from input embeddings by multiplication with 3 different weight matrices

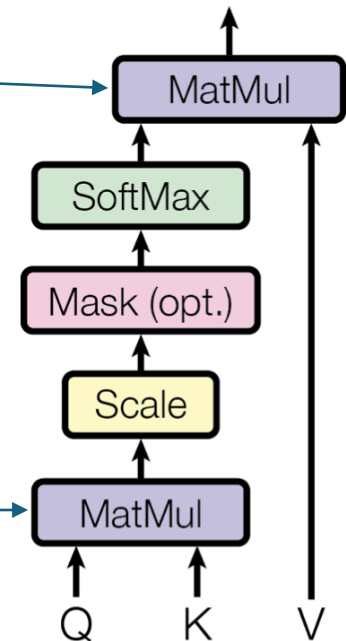
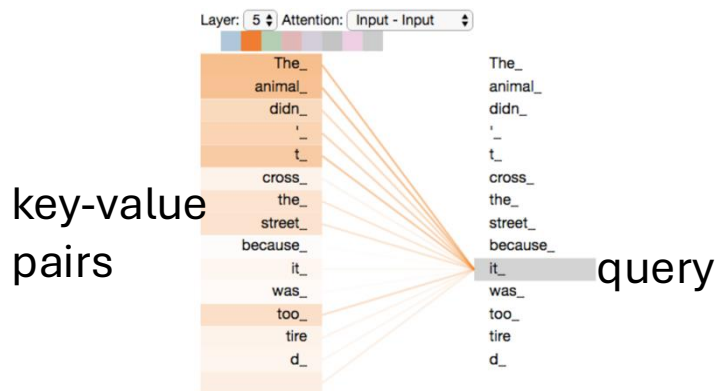
- query Q
- key K
- value V

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

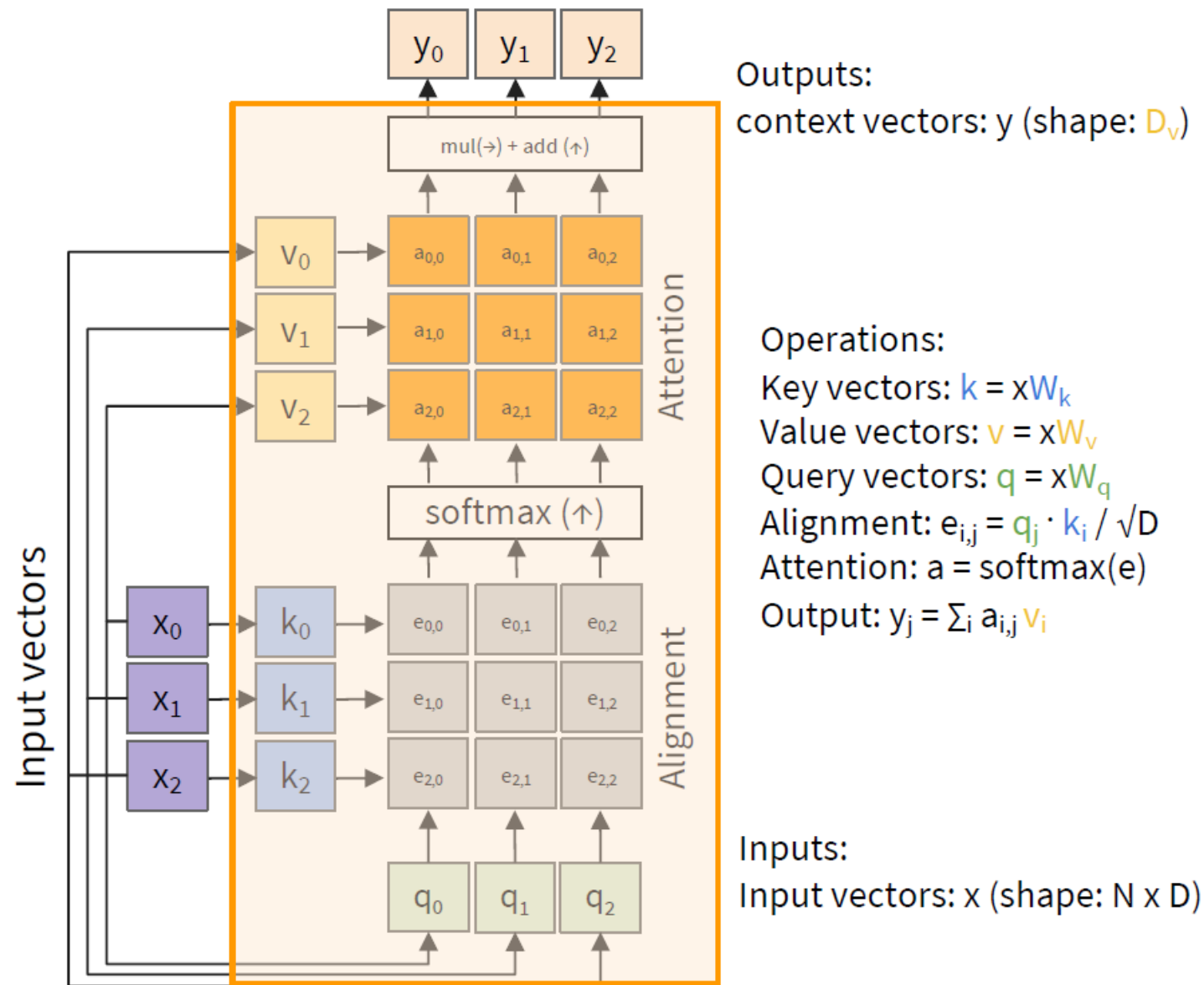
Scaled Dot-Product Attention

filtering: multiplication of attention probabilities with corresponding key-word values

scoring each of the key words (context) with respect to current query word: multiplication of inputs (in contrast to inputs times weights in neural networks)

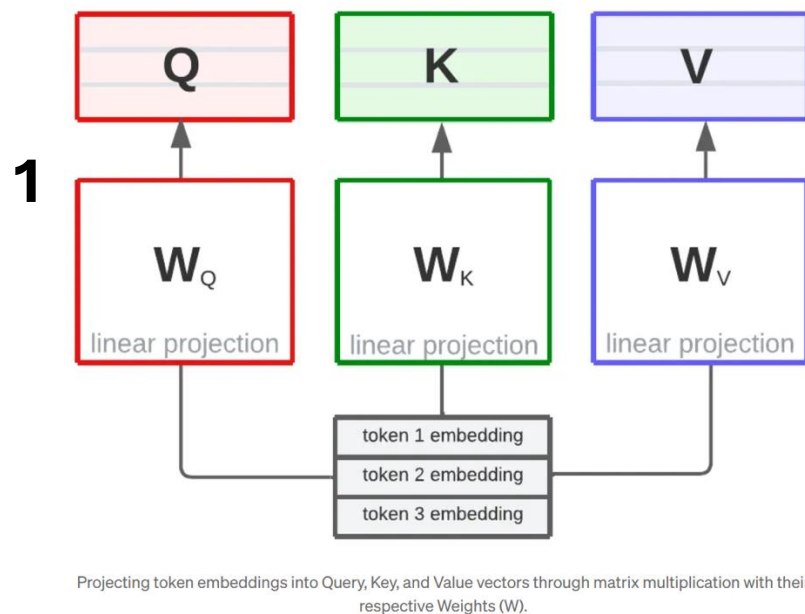


MatMul

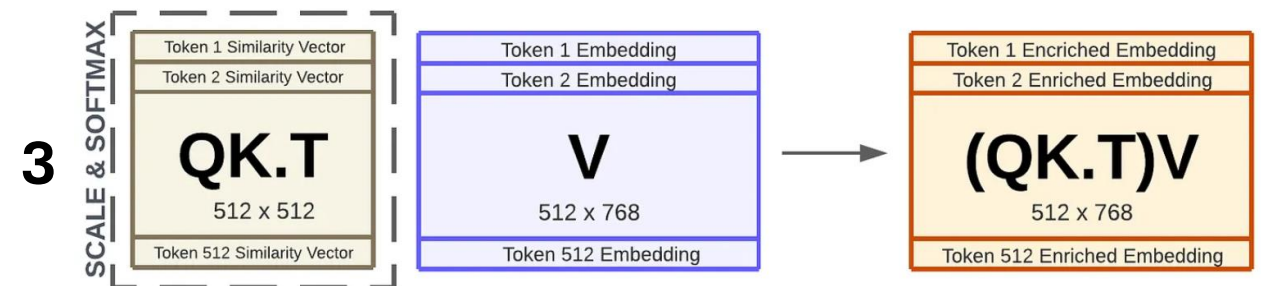
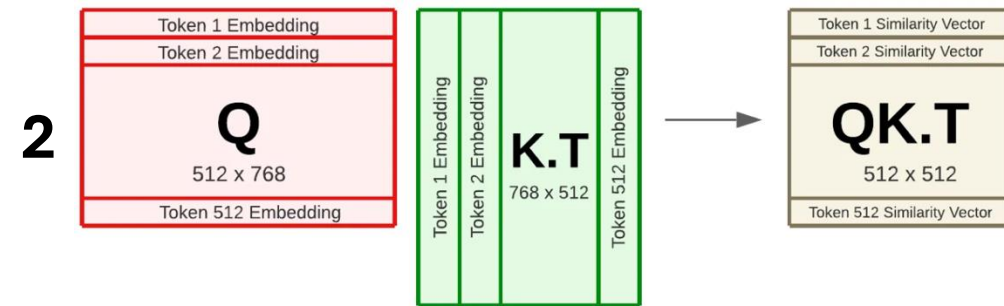


weighted average: reflecting to what degree a token is paying attention to the other tokens in the sequence

Example: Dimensions in BERT

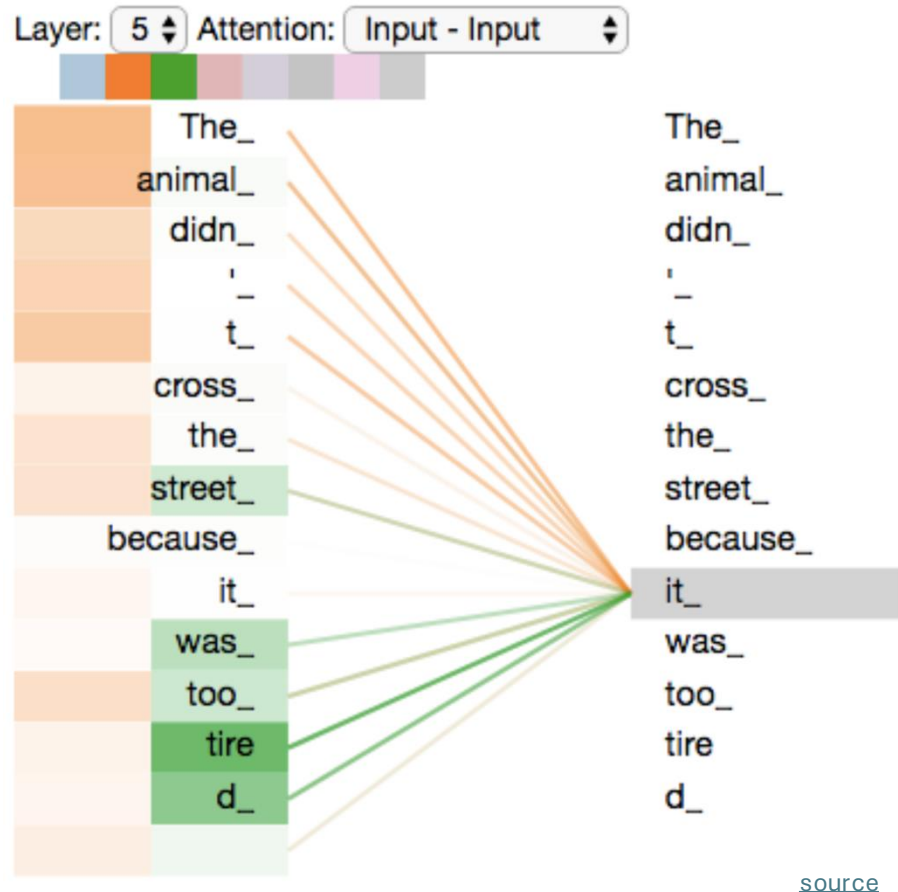


[source](#)

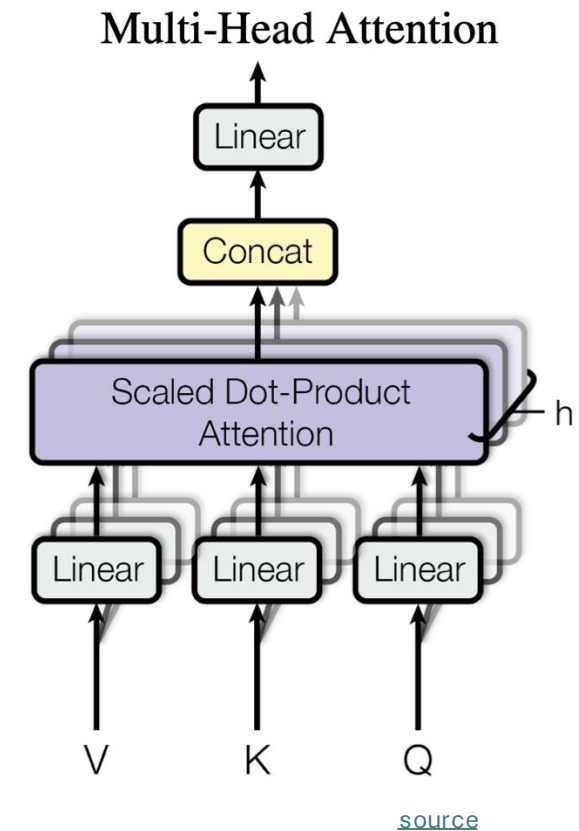


Multi-Head Attention

multiple heads: several attention layers running in parallel



different heads can pay attention to different aspects of input (multiple representation sub-spaces)



Positional Encoding

attention permutation invariant → need for positional encoding to learn from order of sequence

different options:

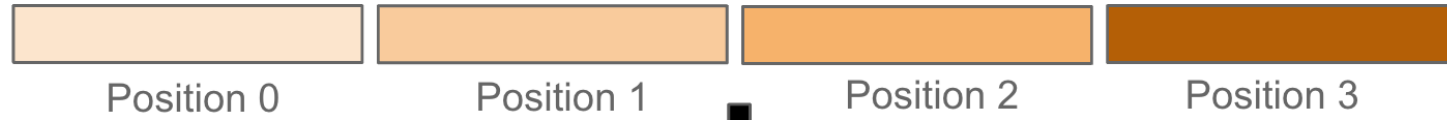
- for each absolute position (from 1 to maximum sequence length), add a learned vector (of dimension d_{model}) to input embeddings → each positional embedding independent of others
- add fixed sinusoidal functions for each position and dimension i

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \quad PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

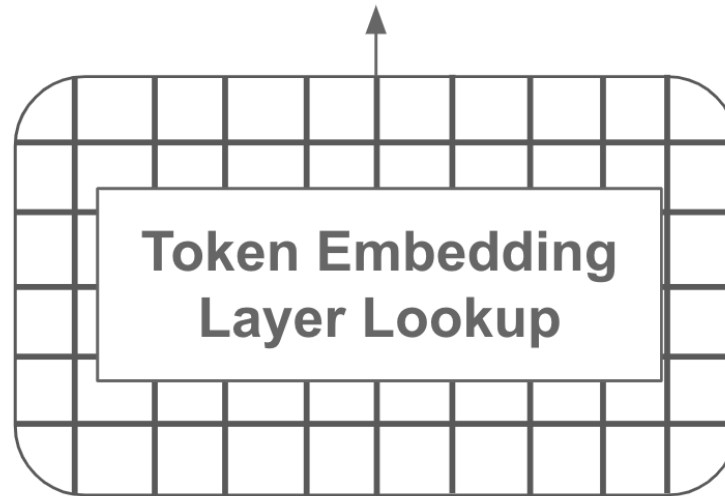
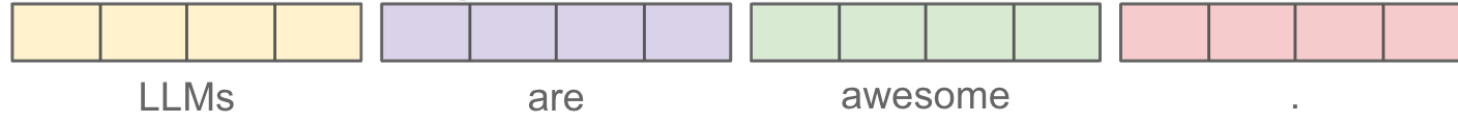
- rotate input embeddings by multiples (position) of a small angle ([RoPE](#)) → captures both relative and absolute positions


$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

Position Embeddings



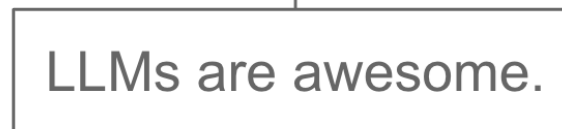
Token Embeddings



Tokenized Text



Raw Text



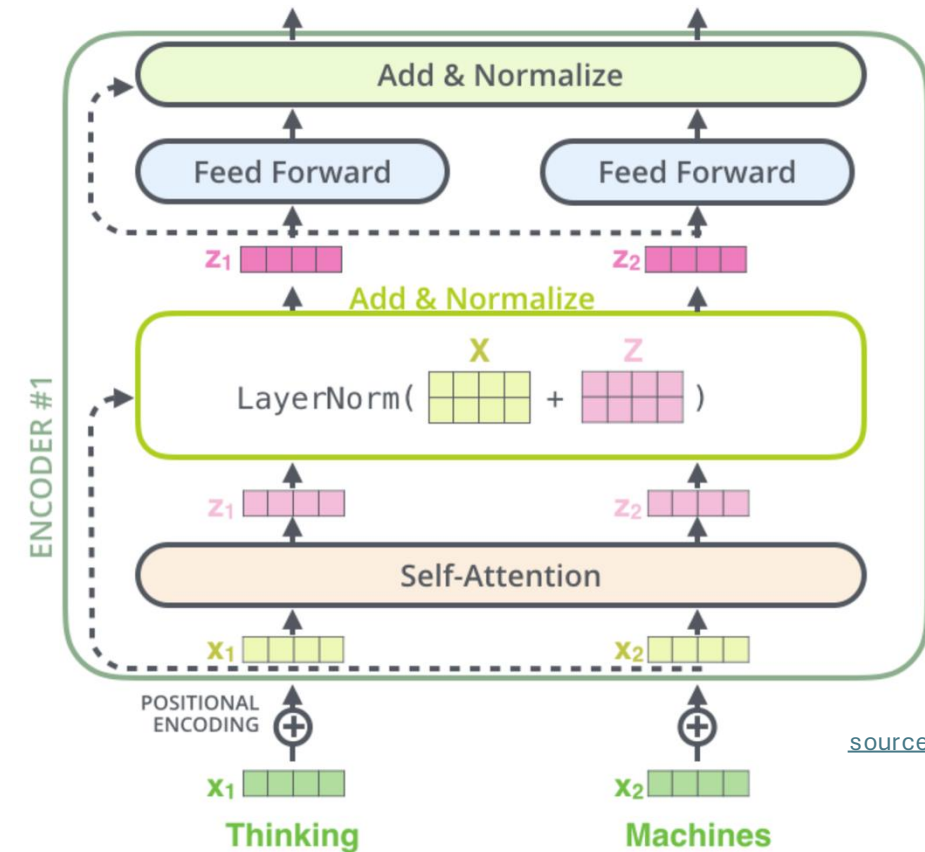
[source](#)

Skip Connections and Layer Normalization

skip connections improve robustness by

- preserving original input (attention layers as filters) as well as gradients (mitigate vanishing-gradient problem)
- easier learning of identity functions (useful for disregarding modules that do not improve model performance)

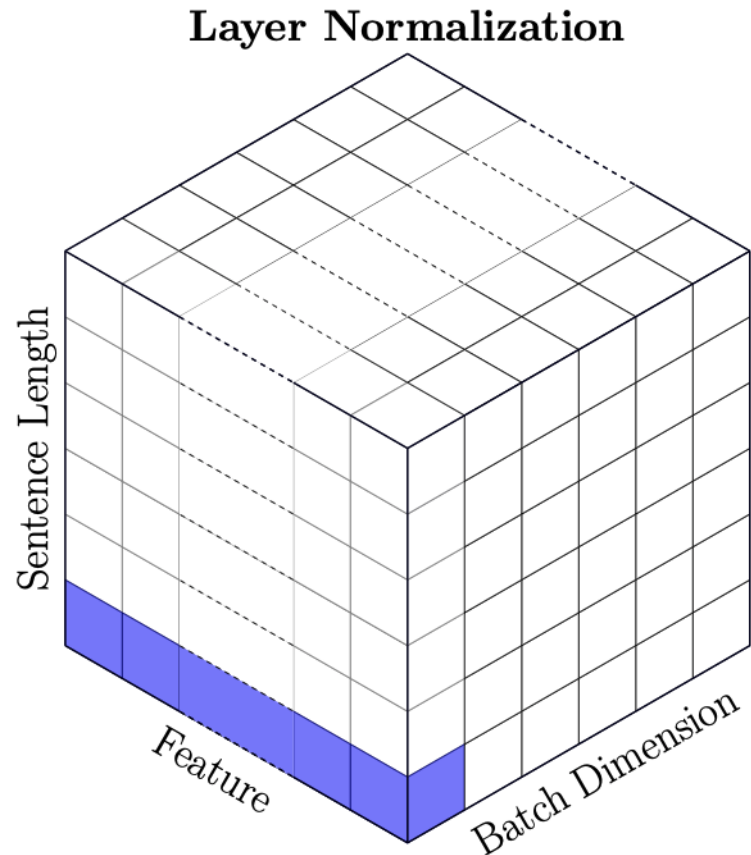
layer normalization improves stability, convergence, and generalization by normalizing activations per input token



Batch vs Layer Norm

batch norm typically in computer vision

layer norm in NLP (variable-sized inputs)



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

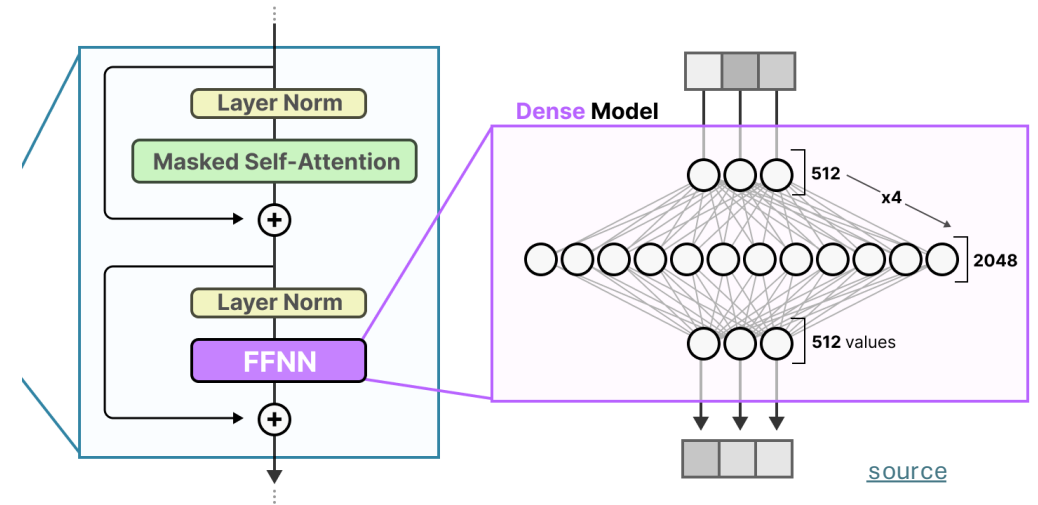
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Role of Feed-Forward Neural Networks (FFNN)

position-wise FFNNs:

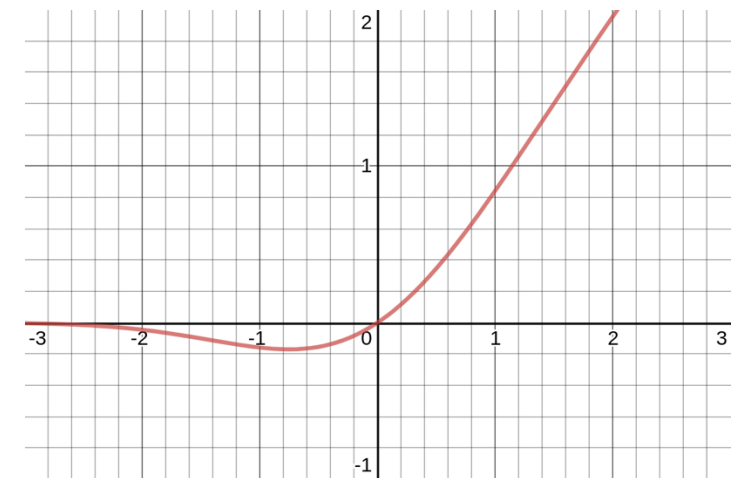
following each attention layer, apply an identical FFNN (with two layers) independently to each embedding vector (correspond to bulk of overall parameters)



can be interpreted as compute after connect (attention)

attention is just weighted averaging

→ need for non-linearities (often GeLU activations) to capture more complex patterns

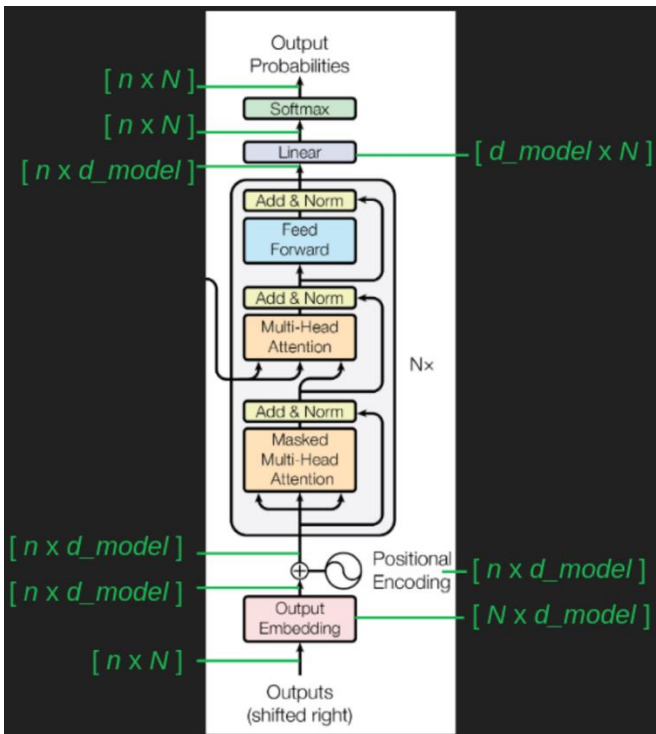


De-Embedding and Softmax

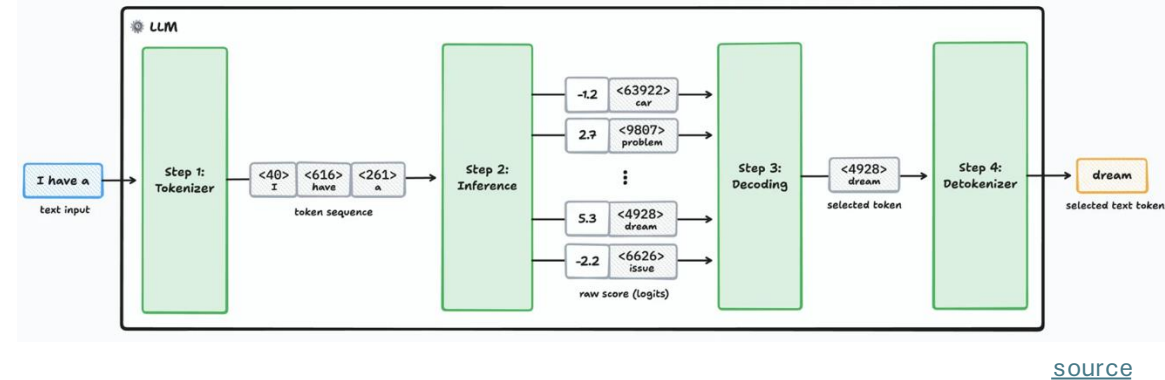
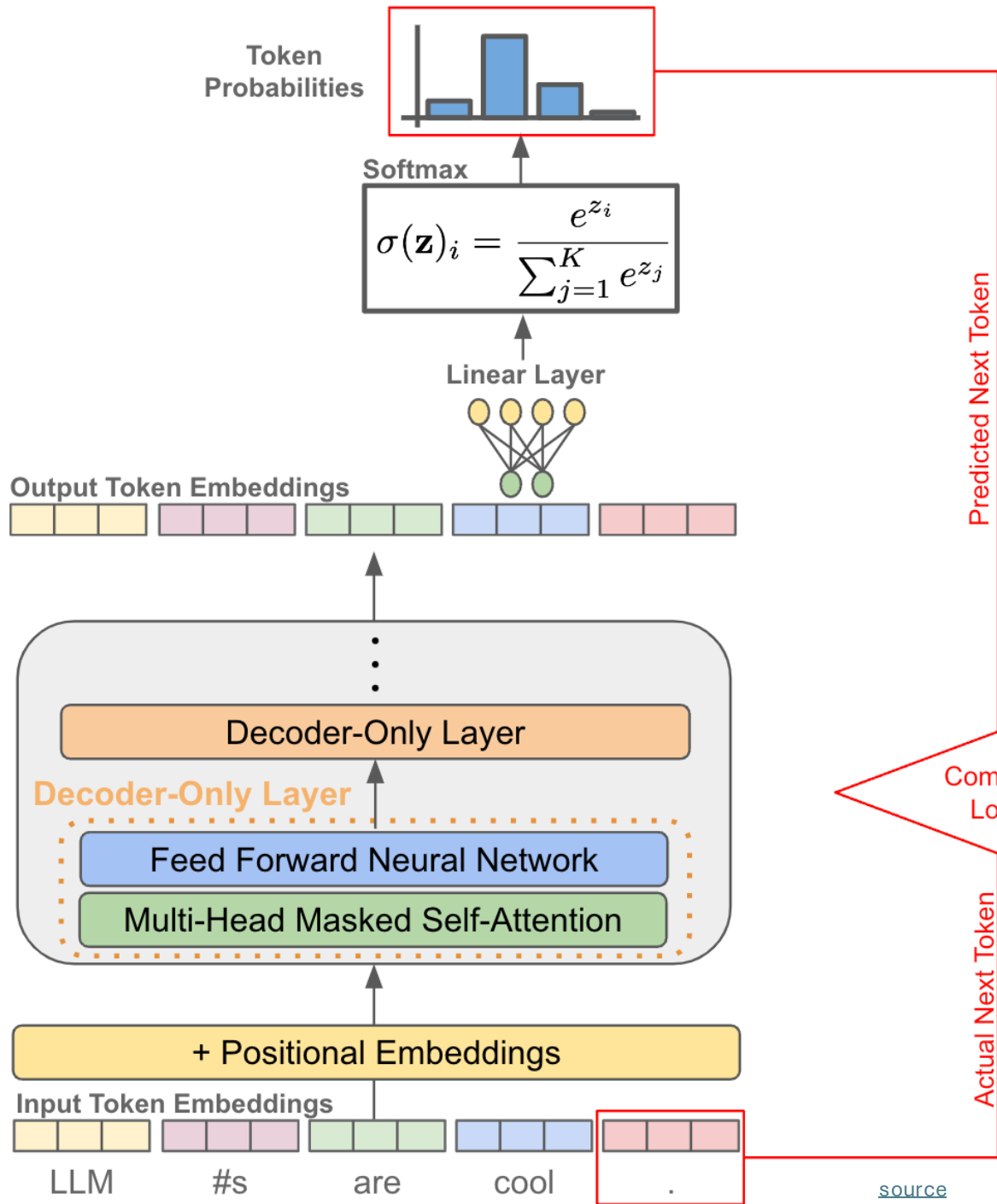
conversion of final decoder output to predicted next-token probabilities for output vocabulary

- de-embedding: linear transformation (fully-connected network layer) to vocabulary size
- softmax: transformation to probabilities (“softness” can be controlled by hyperparameter $\rightarrow$ temperature)

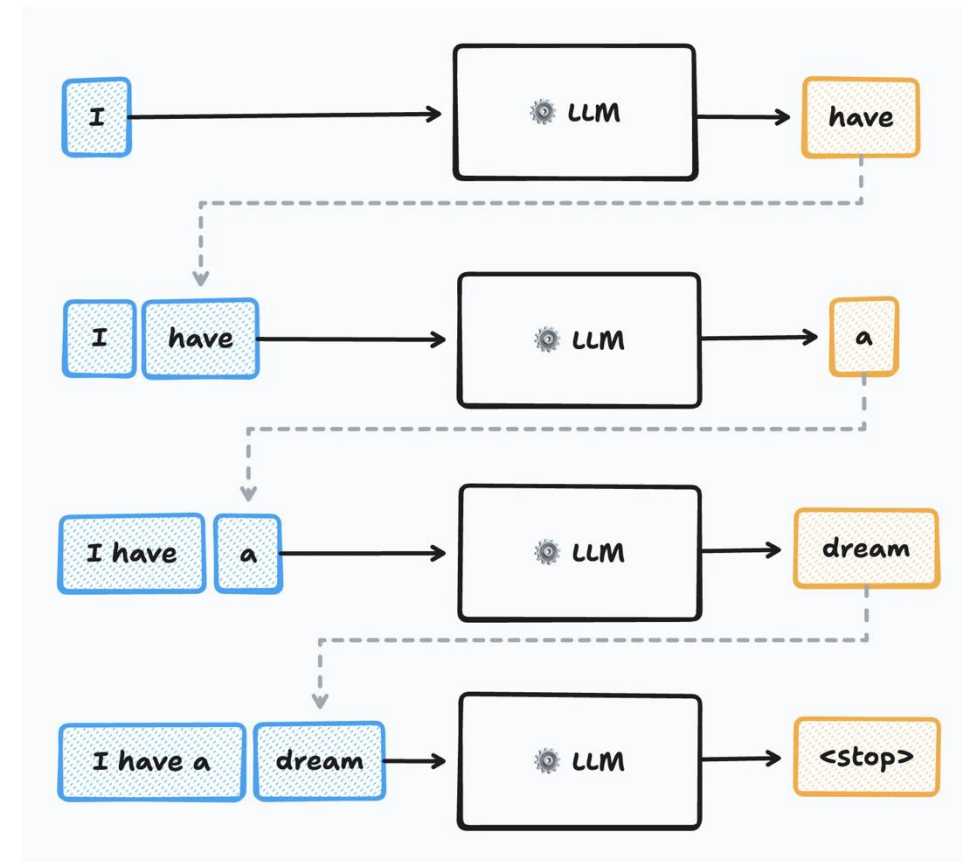
n : sequence length
 N : vocabulary size
 d_{model} : embedding dimensions



Decoder-Only Transformer



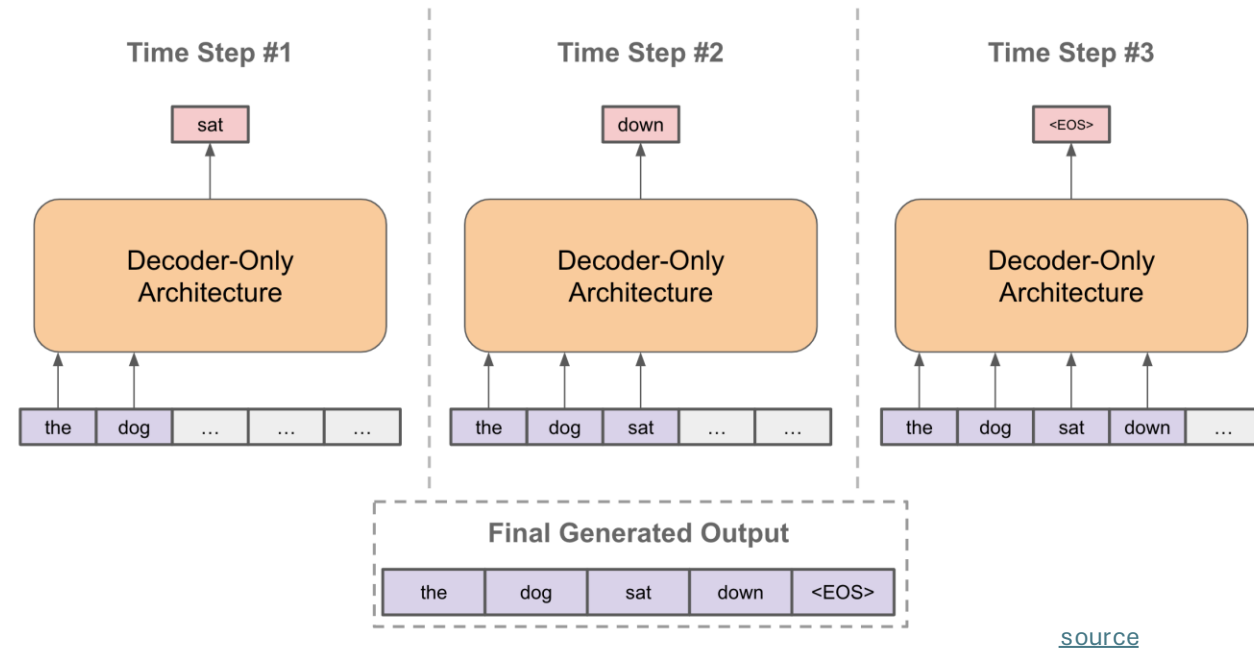
[source](#)



Sequence Completion

autoregressive: for each step (iteratively), choose one output token and then add it to decoder input sequence for next step

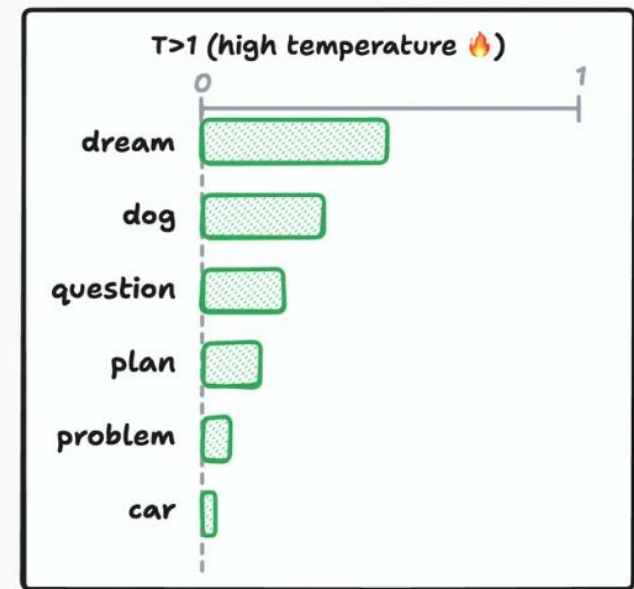
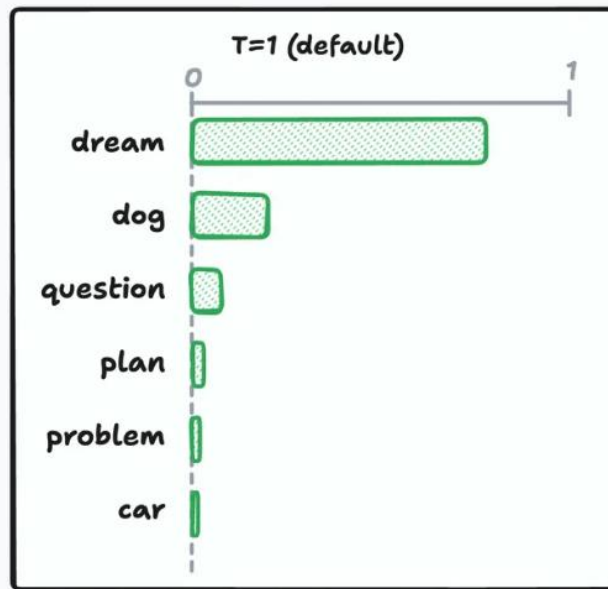
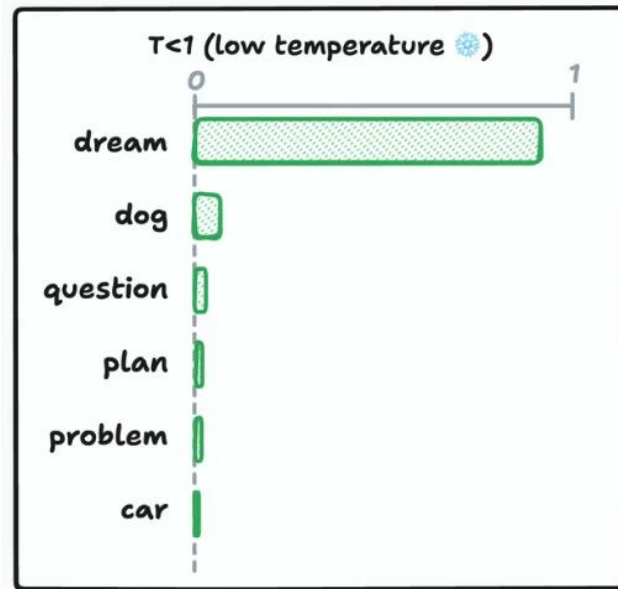
typically, pick according to softmax probabilities (degree of randomness controlled by softmax temperature)



prompt: externally given initial sequence for running start and context on which to build rest of sequence

Temperature Parameter

$$\text{softmax}(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^n e^{z_j/T}}$$

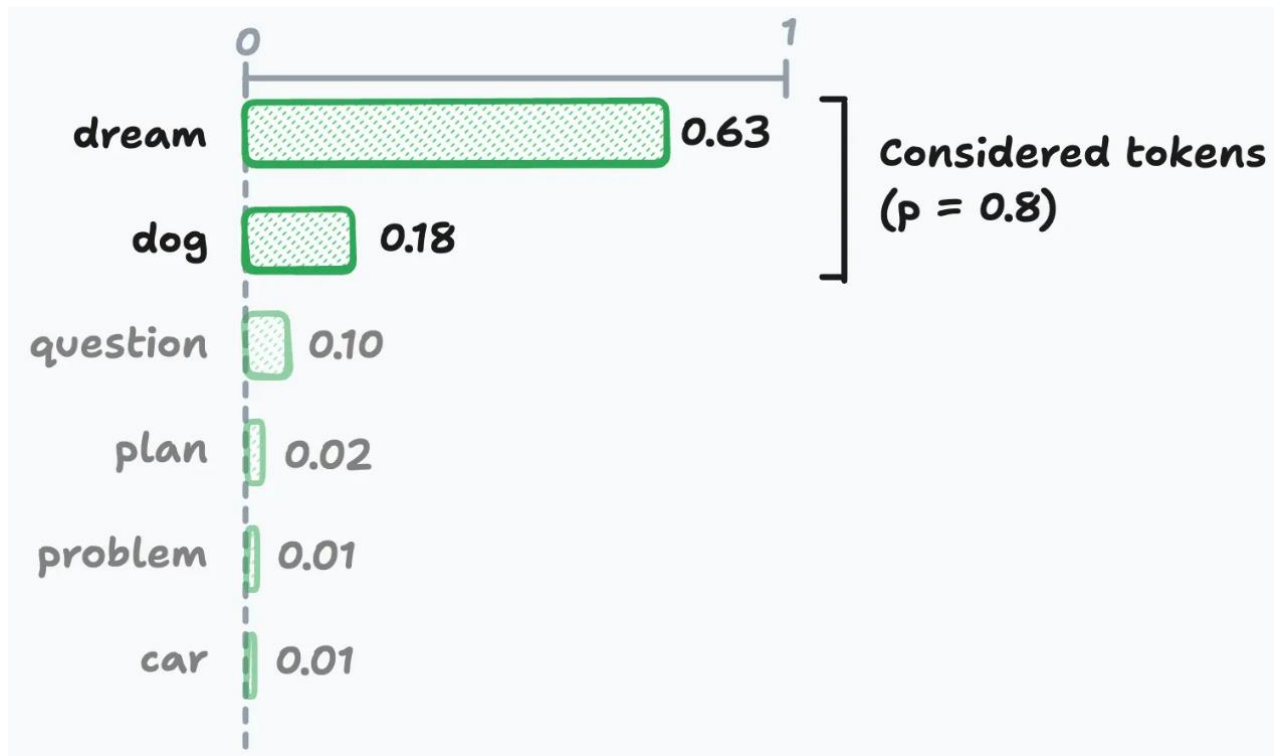
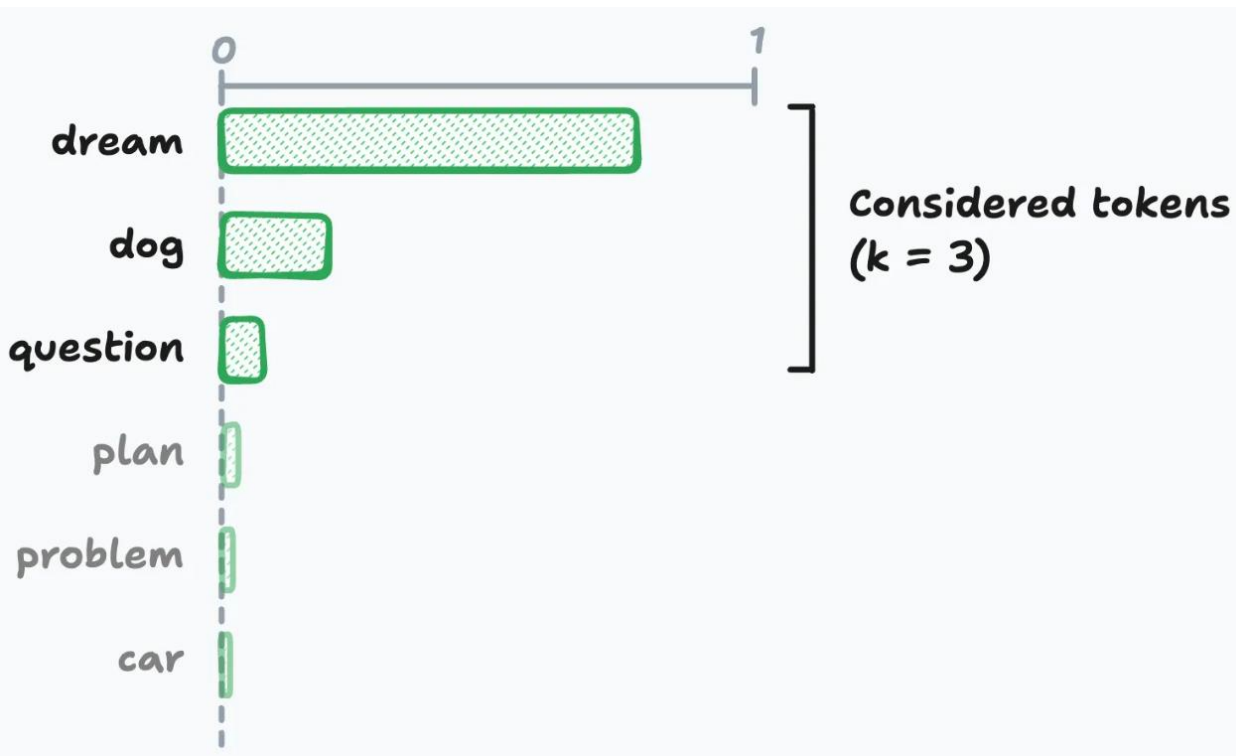


[source](#)

$T \rightarrow 0 \rightarrow$ greedy

creativity ;)

top-k & top-p Sampling



[source](#)